

Universidad Católica del Uruguay  
Facultad de Ingeniería y Tecnologías

---

# Efectos del error numérico en la implementación de sumatorias

---

Curso: Cálculo Aplicado

Autor: Stefano Francolino  
Segundo integrante: [Nombre del segundo integrante o trabajo individual]  
Fecha de entrega: [Fecha de entrega]

Repositorio público: [github.com/Stefano936/Tarea-Error-Numerico](https://github.com/Stefano936/Tarea-Error-Numerico)

## Resumen

Este trabajo analiza cómo la precisión finita altera sumatorias que son equivalentes en aritmética real. Se realizaron tres experimentos reproducibles en Python. Primero se compararon tres asociaciones de  $1 + b^k - b^k$  con bases enteras, flotantes, negativas y de módulo no mayor que uno, además de contrastar los enteros arbitrarios de Python con representaciones fijas. Luego se sumó  $1/[k(k+1)]$  de mayor a menor módulo, de menor a mayor, en orden aleatorio y con el algoritmo de Kahan, usando los dos rangos de  $N$  indicados en la consigna. Finalmente se estudiaron representaciones equivalentes de  $b_N$  y  $c_N$ , junto con sus formas cerradas y términos individuales.

Los enteros de Python preservaron  $a_N = N$ , mientras que en `float64` las asociaciones exteriores perdieron la unidad cuando  $b^k$  superó la resolución disponible. Las potencias desbordadas produjeron infinito y luego `NaN`; con bases negativas se intercambió el comportamiento de las asociaciones exteriores. Para  $N = 10^6$ , el error relativo fue  $4,76 \times 10^{-14}$  al sumar de mayor a menor, frente a cero en el redondeo de referencia para el orden inverso y Kahan. Treinta permutaciones aleatorias produjeron errores entre  $2,22 \times 10^{-15}$  y  $6,13 \times 10^{-14}$ . En  $c_N$ , la forma sustractiva acumuló una diferencia de  $2,85 \times 10^{-11}$  para  $N = 10^4$  y su primer término nulo apareció en  $k = 2^{26}$ . Se concluye que la equivalencia algebraica no garantiza equivalencia numérica: el tipo, el orden y la forma de evaluar una expresión determinan la información que se conserva.

# Índice

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                   | <b>i</b>  |
| <b>1. Introducción</b>                                           | <b>1</b>  |
| <b>2. Marco teórico</b>                                          | <b>1</b>  |
| 2.1. Representación finita, <b>binary64</b> y redondeo . . . . . | 1         |
| 2.2. Asociatividad y orden de suma . . . . .                     | 1         |
| 2.3. Error absoluto y relativo . . . . .                         | 2         |
| 2.4. Suma compensada de Kahan . . . . .                          | 2         |
| 2.5. Cancelación catastrófica . . . . .                          | 2         |
| <b>3. Metodología</b>                                            | <b>2</b>  |
| 3.1. Entorno y reproducibilidad . . . . .                        | 2         |
| 3.2. Asociatividad . . . . .                                     | 3         |
| 3.3. Orden de sumación . . . . .                                 | 3         |
| 3.4. Representaciones equivalentes . . . . .                     | 3         |
| <b>4. Resultados</b>                                             | <b>4</b>  |
| 4.1. Asociatividad con enteros y flotantes . . . . .             | 4         |
| 4.2. Influencia del orden de suma . . . . .                      | 5         |
| 4.3. Representaciones de $b_N$ . . . . .                         | 7         |
| 4.4. Representaciones de $c_N$ . . . . .                         | 8         |
| <b>5. Discusión</b>                                              | <b>10</b> |
| 5.1. Pérdida de la unidad, infinito y NaN . . . . .              | 10        |
| 5.2. Bases negativas, $b \leq 1$ y enteros fijos . . . . .       | 10        |
| 5.3. Orden, Kahan y variabilidad aleatoria . . . . .             | 11        |
| 5.4. Representaciones de $b_N$ . . . . .                         | 11        |
| 5.5. Cancelación en $c_N$ . . . . .                              | 11        |
| 5.6. Limitaciones . . . . .                                      | 12        |
| <b>6. Conclusiones</b>                                           | <b>12</b> |

## 1. Introducción

Las sumatorias aparecen en promedios, probabilidades, simulaciones y algoritmos científicos. En el modelo matemático, sus términos pertenecen a los números reales y pueden reagruparse sin cambiar el resultado. Una computadora, en cambio, opera sobre un conjunto finito de valores. Cada resultado intermedio debe representarse en el formato disponible y puede requerir redondeo. Por eso, una igualdad correcta en el papel puede dar lugar a implementaciones con distinta precisión.

El problema tiene importancia directa en informática. Un programa puede ejecutar sin errores de sintaxis y aun así perder contribuciones pequeñas, desbordar el rango numérico o amplificar redondeos al restar cantidades cercanas. Analizar el error numérico exige entonces observar tanto el resultado final como la secuencia de operaciones que lo produce.

El objetivo general es estudiar experimentalmente el efecto de la representación finita en la implementación de sumatorias. Los objetivos específicos son: (i) comparar tres asociaciones de una misma expresión con tipos enteros y flotantes; (ii) medir la influencia del orden de acumulación y la mejora de Kahan; (iii) determinar si distintas representaciones de  $b_N$  y  $c_N$  son numéricamente equivalentes; y (iv) explicar los resultados a partir del redondeo, el desbordamiento y la cancelación catastrófica. El informe presenta primero el marco teórico y la metodología, luego separa los resultados observados de su discusión y termina con las conclusiones.

## 2. Marco teórico

### 2.1. Representación finita, `binary64` y redondeo

La norma IEEE 754 define formatos binarios y decimales, operaciones, reglas de redondeo y condiciones excepcionales para la aritmética de punto flotante ([Institute of Electrical and Electronics Engineers, 2019](#)). En la plataforma utilizada, `float` y `numpy.float64` siguen el formato `binary64`: el significando dispone de 53 bits de precisión, incluyendo el bit implícito. Esto equivale a unas 15 o 16 cifras decimales significativas, pero no significa que toda fracción con esa cantidad de cifras sea representable.

La mayoría de las fracciones decimales no tiene una expansión binaria finita. Por lo tanto, el valor almacenado es el representable más cercano y cada operación puede introducir un nuevo redondeo ([Goldberg, 1991](#); [Python Software Foundation, 2026b](#)). El épsilon de máquina usado en los experimentos fue

$$\varepsilon = 2^{-52} = 2,220446049250313 \times 10^{-16}. \quad (1)$$

La unidad en el último lugar (ulp) es la separación entre números representables consecutivos en una región. Esa separación aumenta con la magnitud. Alrededor de  $2^{53}$ , por ejemplo, los `float64` consecutivos ya están separados por dos unidades; por eso  $2^{53} + 1$  no se distingue de  $2^{53}$ .

### 2.2. Asociatividad y orden de suma

En los números reales se cumple  $(x + y) + z = x + (y + z)$ . En punto flotante cada operación intermedia se redondea, de modo que en general

$$\text{fl}(\text{fl}(x + y) + z) \neq \text{fl}(x + \text{fl}(y + z)). \quad (2)$$

La suma binaria continúa siendo conmutativa bajo el mismo modo de redondeo, pero reordenar tres o más términos cambia las asociaciones implícitas. En una secuencia positiva con términos de magnitudes diferentes, sumar primero los menores suele preservar mejor su contribución, porque el acumulador inicial tiene una resolución comparable ([Higham, 2002](#)).

### 2.3. Error absoluto y relativo

Para una aproximación  $x^{\text{num}}$  y una referencia no nula  $x^{\text{ref}}$ , se usan

$$E_{\text{abs}} = |x^{\text{num}} - x^{\text{ref}}|, \quad (3)$$

$$E_{\text{rel}} = \frac{|x^{\text{num}} - x^{\text{ref}}|}{|x^{\text{ref}}|}. \quad (4)$$

El primero conserva las unidades del problema; el segundo permite comparar errores respecto a la escala del resultado. Que  $E_{\text{rel}} = 0$  en `float64` indica coincidencia con la referencia redondeada usada por el programa, no una demostración de exactitud en los reales.

### 2.4. Suma compensada de Kahan

Kahan propuso conservar una compensación para parte de los bits descartados en cada suma ([Kahan, 1965](#)). Con acumulador  $s$ , compensación  $c$  y término  $x_k$ , la implementación utilizada es

$$y = x_k - c, \quad t = s + y, \quad (5)$$

$$c = (t - s) - y, \quad s = t. \quad (6)$$

El algoritmo no vuelve exacta la aritmética, pero reduce la acumulación de redondeos con un costo constante por término.

### 2.5. Cancelación catastrófica

La cancelación catastrófica ocurre al restar números cercanos que ya contienen errores pequeños. Los dígitos principales se eliminan y el resultado depende de los dígitos menos confiables restantes ([Goldberg, 1991](#)). Las expresiones

$$\sqrt{k^2 + 1} - k \quad \text{y} \quad \frac{1}{\sqrt{k^2 + 1} + k} \quad (7)$$

son algebraicamente iguales, pero la segunda evita una sustracción entre cantidades cercanas y por eso es más estable.

## 3. Metodología

### 3.1. Entorno y reproducibilidad

Los experimentos se ejecutaron en Windows 11 de 64 bits con Python 3.14.3, NumPy 2.3.5 y Matplotlib 3.10.8. Los cálculos flotantes se realizaron en `float64`; el épsilon de máquina fue el

de la Ecuación 1. El código, los CSV, las pruebas y las figuras están disponibles en [el repositorio público del proyecto](#).

No se utilizó la función incorporada `sum`. Las funciones principales recorren los términos con acumuladores explícitos y reciben solo  $N$ , salvo la función de asociatividad, que también requiere  $b$ . Para los dos barridos de mil valores se empleó `numpy.add.accumulate` como auxiliar de rendimiento. Las pruebas compararon ese auxiliar con los bucles explícitos para ambos órdenes y confirmaron igualdad bit a bit en los casos evaluados.

Las semillas fueron 20260912 para el primer barrido aleatorio, 20260913 para el segundo y 20260914 a 20260943 para las 30 repeticiones. El generador es determinista bajo el entorno documentado ([Python Software Foundation, 2026c](#)). Los errores originales iguales a cero permanecen como cero en los CSV. Solo en las figuras logarítmicas se reemplazaron visualmente por  $5 \times 10^{-18}$ .

### 3.2. Asociatividad

Se estudió

$$a_N = \sum_{k=1}^N (1 + b^k - b^k) = N \quad (8)$$

con las asociaciones  $(1 + b^k) - b^k$ ,  $1 + (b^k - b^k)$  y  $(1 - b^k) + b^k$ . Se generaron curvas para  $N = 1, \dots, 1000$  con  $b \in \{2, 3, 5, 10\}$  como `int` y como `float`. Se repitió el estudio con bases negativas y con  $b \in \{-1, -0.5, 0, 0.5, 1\}$ .

Se registró la primera desviación respecto de  $a_N = N$  y el último índice finito. Como bonus, se comparó el entero de precisión arbitraria de Python con `int8`, `int32` e `int64`, cuyos límites fijos pueden producir desbordamiento modular ([Python Software Foundation, 2026a](#); [NumPy Developers, 2026](#)).

### 3.3. Orden de sumación

La sucesión elegida fue

$$b_N = \sum_{k=1}^N \frac{1}{k(k+1)} = \frac{N}{N+1}. \quad (9)$$

Se implementaron suma de mayor a menor módulo (orden  $k = 1, \dots, N$ ), de menor a mayor ( $k = N, \dots, 1$ ), una permutación aleatoria y Kahan en orden natural. El error se calculó con la Ecuación 4. Los rangos fueron exactamente  $N = 10, 20, \dots, 10000$  y  $N = 1000, 2000, \dots, 1000000$ . Para  $N = 10^6$  se realizaron 30 permutaciones independientes y se calcularon mínimo, mediana y máximo del error.

### 3.4. Representaciones equivalentes

Para  $N = 1, 10, 20, \dots, 10000$  se compararon

$$b_N^{(1)} = \sum_{k=1}^N \frac{1}{k(k+1)}, \quad b_N^{(2)} = \sum_{k=1}^N \left( \frac{1}{k} - \frac{1}{k+1} \right) \quad (10)$$

respecto de  $N/(N+1)$ . También se evaluaron las formas cerradas  $1 - 1/(N+1)$  y  $N/(N+1)$  en una grilla logarítmica hasta  $10^{18}$  y alrededor de potencias de dos, en especial  $2^{53}$ .

Para la segunda sucesión se implementaron

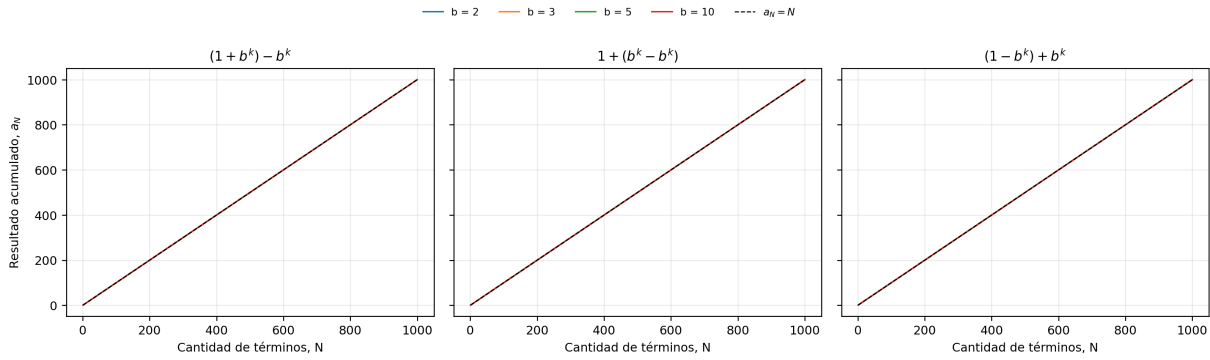
$$c_N^{(1)} = \sum_{k=1}^N \frac{1}{\sqrt{k^2 + 1} + k}, \quad c_N^{(2)} = \sum_{k=1}^N (\sqrt{k^2 + 1} - k) \quad (11)$$

y se calculó  $D_N = |c_N^{(1)} - c_N^{(2)}|$ . Finalmente se buscaron los primeros enteros cuyo error relativo sustractivo alcanzó  $10^{-12}$ ,  $10^{-8}$  y  $10^{-4}$ , y el primer término redondeado a cero.

## 4. Resultados

### 4.1. Asociatividad con enteros y flotantes

Con `int`, las 12 combinaciones de base y asociación coincidieron con  $a_N = N$  en todo el rango. Las curvas superpuestas de la Figura 1 muestran esa coincidencia.



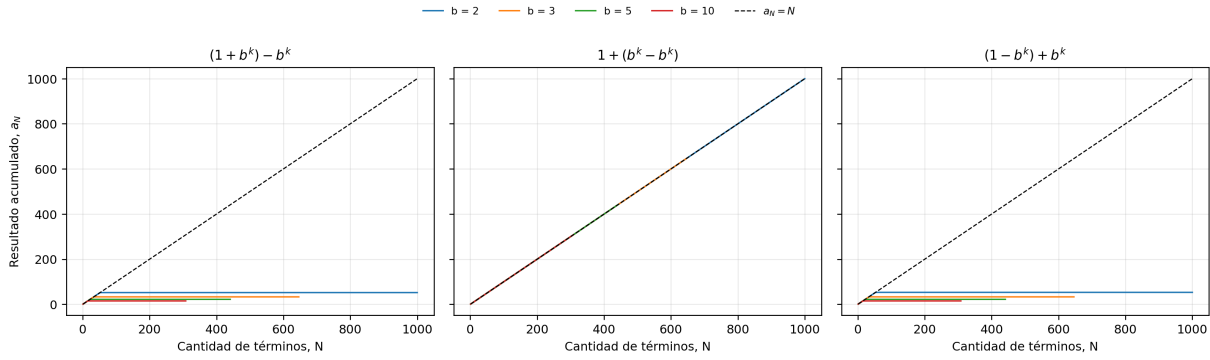
**Figura 1:** Tres asociaciones de  $a_N$  con  $b = 2, 3, 5, 10$  de tipo `int` y  $N = 1, \dots, 1000$ . Todas las curvas coinciden con la referencia discontinua  $a_N = N$ .

Con `float`, las asociaciones exteriores dejaron de crecer antes del desbordamiento. La asociación central permaneció sobre la referencia mientras  $b^k$  fue finito. La Tabla 1 y la Figura 2 resumen el comportamiento.

**Cuadro 1:** Primera desviación y último  $N$  finito con bases positivas `float`.

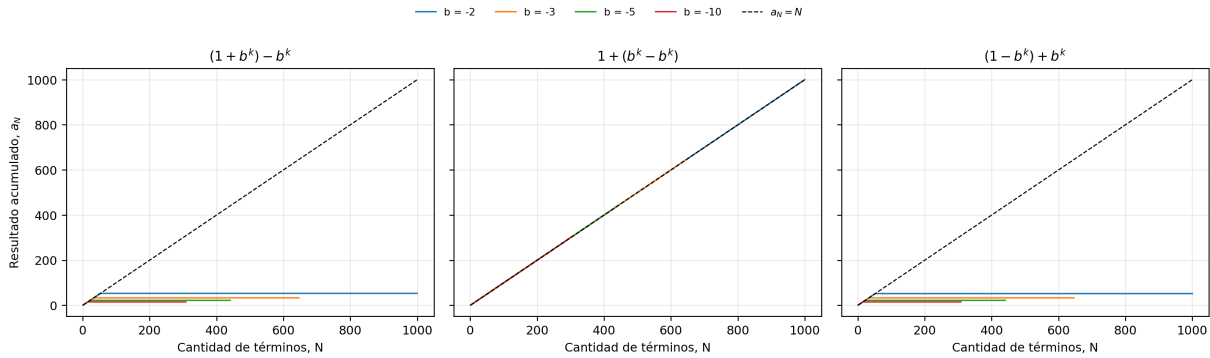
| $b$  | $(1 + b^k) - b^k$ | $1 + (b^k - b^k)$ | $(1 - b^k) + b^k$ | Último finito |
|------|-------------------|-------------------|-------------------|---------------|
| 2,0  | 53                | —                 | 54                | 1000          |
| 3,0  | 34                | 647               | 34                | 646           |
| 5,0  | 23                | 442               | 23                | 441           |
| 10,0 | 16                | 309               | 16                | 308           |

*Nota.* El guión indica que no hubo desviación en el rango. En la asociación central, la primera desviación coincide con la aparición de `NaN`. Fuente: `datos/asociatividad.csv`.



**Figura 2:** Tres asociaciones de  $a_N$  con bases positivas `float`. Las interrupciones de las curvas para  $b = 3, 5, 10$  corresponden a valores no finitos, no a datos omitidos.

Para  $b = -2$ , los resultados finales de las asociaciones exteriores fueron 53 y 52, en orden inverso al caso  $b = 2$  (52 y 53). Las demás bases negativas crecientes en módulo también reprodujeron las desviaciones y los límites finitos de sus valores absolutos (Figura 3). Para  $-1 \leq b \leq 1$ , las tres asociaciones dieron  $a_{1000} = 1000$ .



**Figura 3:** Tres asociaciones de  $a_N$  con  $b = -2, -3, -5, -10$  de tipo `float`. La alternancia de signo intercambia cuál asociación exterior conserva la unidad en el umbral para  $|b| = 2$ .

La Tabla 2 presenta el bonus de enteros fijos. Python conservó el resultado y representó  $10^{1000}$  con 3322 bits. Los tipos fijos no pudieron representar esa potencia; el acumulador `int8` además desbordó al intentar almacenar 1000 y terminó en  $-24$ .

**Cuadro 2:** Resultado para  $N = 1000$ ,  $b = 10$  con enteros de distinta capacidad.

| Tipo                    | Bits     | $A_N$ | $B_N$ | $C_N$ |
|-------------------------|----------|-------|-------|-------|
| Python <code>int</code> | variable | 1000  | 1000  | 1000  |
| <code>int8</code>       | 8        | -24   | -24   | -24   |
| <code>int32</code>      | 32       | 1000  | 1000  | 1000  |
| <code>int64</code>      | 64       | 1000  | 1000  | 1000  |

*Nota.* Las potencias intermedias de los tipos NumPy se calcularon con aritmética modular. El resultado 1000 cabe en 32 y 64 bits; no cabe en 8 bits. Fuente: `datos/enteros_fijos.csv`.

## 4.2. Influencia del orden de suma

En el rango corto, los errores se mantuvieron entre cero y unos pocos múltiplos de  $10^{-15}$  (Figura 4). En el rango largo, el orden natural y el aleatorio alcanzaron el orden de  $10^{-14}$ , mientras

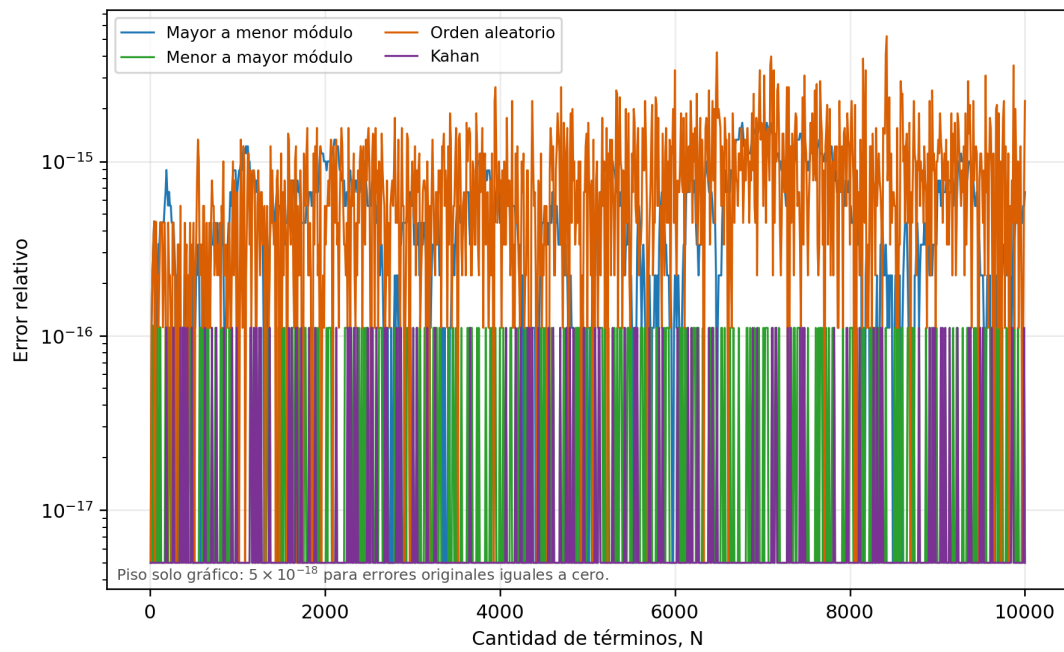
el orden inverso y Kahan quedaron en cero o cerca de  $10^{-16}$  (Figura 5).

**Cuadro 3:** Error relativo de los cuatro métodos en valores representativos de  $N$ .

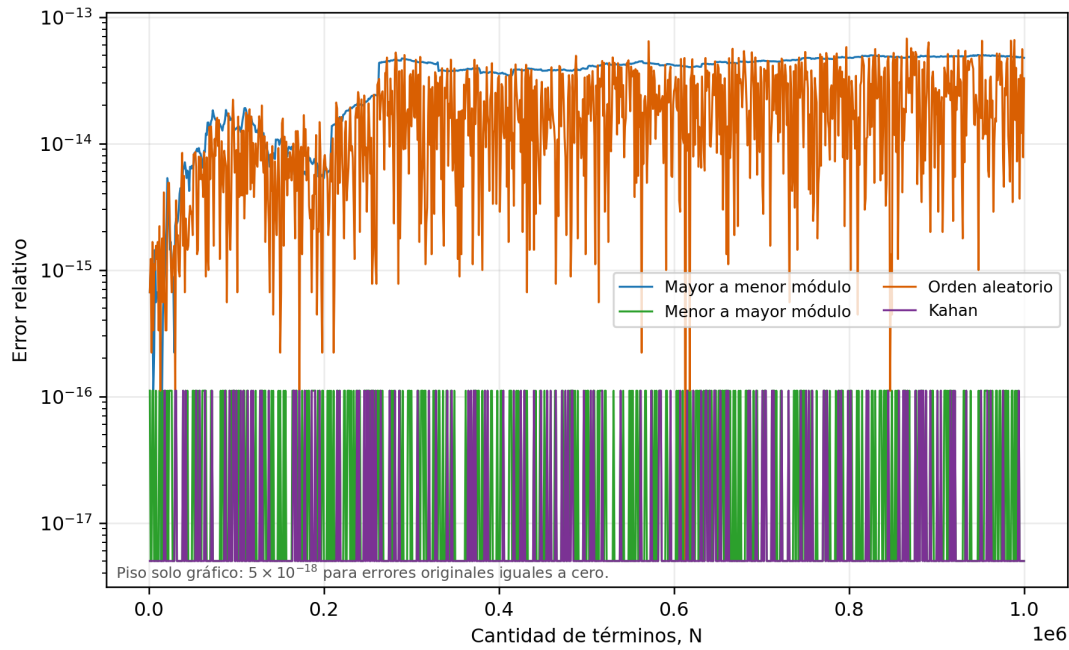
| $N$     | Mayor a menor          | Menor a mayor          | Aleatoria              | Kahan                  |
|---------|------------------------|------------------------|------------------------|------------------------|
| 10      | 0                      | 0                      | 0                      | 0                      |
| 100     | $3,36 \times 10^{-16}$ | 0                      | $1,12 \times 10^{-16}$ | 0                      |
| 1000    | $6,67 \times 10^{-16}$ | $1,11 \times 10^{-16}$ | $6,67 \times 10^{-16}$ | 0                      |
| 10000   | $6,66 \times 10^{-16}$ | 0                      | $2,22 \times 10^{-15}$ | 0                      |
| 100000  | $1,31 \times 10^{-14}$ | $1,11 \times 10^{-16}$ | $8,88 \times 10^{-15}$ | $1,11 \times 10^{-16}$ |
| 1000000 | $4,76 \times 10^{-14}$ | 0                      | $3,28 \times 10^{-14}$ | 0                      |

*Nota.* Los ceros son valores originales. La columna aleatoria corresponde a las semillas de cada barrido.

Fuentes: `datos/orden_10_a_10000.csv` y `datos/orden_1000_a_1000000.csv`.

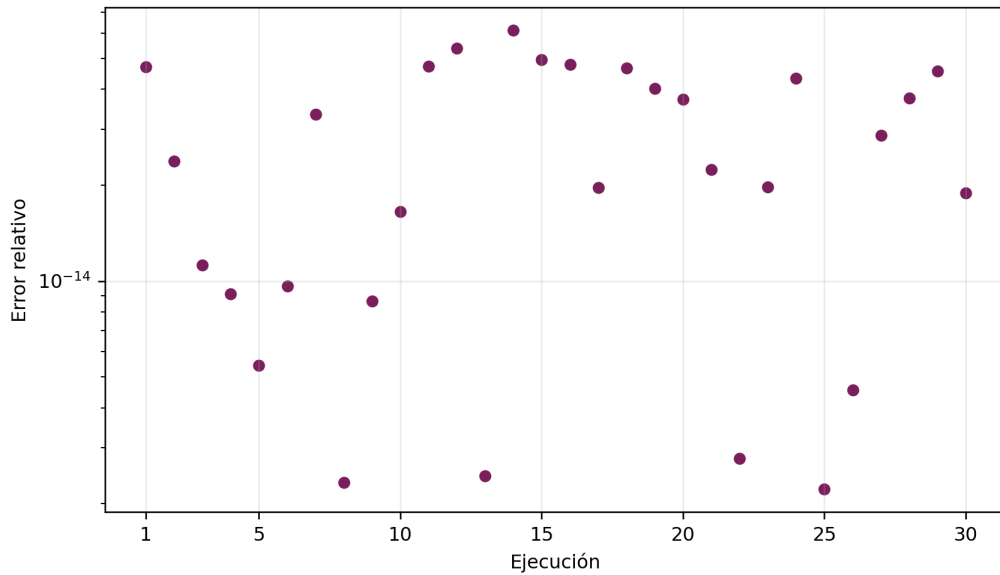


**Figura 4:** Error relativo para  $N = 10, 20, \dots, 10000$ . El eje vertical es logarítmico. Los errores originales iguales a cero se dibujan en  $5 \times 10^{-18}$  solo para hacerlos visibles.



**Figura 5:** Error relativo para  $N = 1000, 2000, \dots, 1000000$ . En la escala logarítmica, el piso  $5 \times 10^{-18}$  se aplica únicamente a la representación gráfica de errores nulos.

Las 30 permutaciones de un millón de términos produjeron 30 errores distintos. El mínimo fue  $2,22 \times 10^{-15}$ , la mediana  $2,31 \times 10^{-14}$  y el máximo  $6,13 \times 10^{-14}$ . La dispersión se aprecia en la Figura 6.

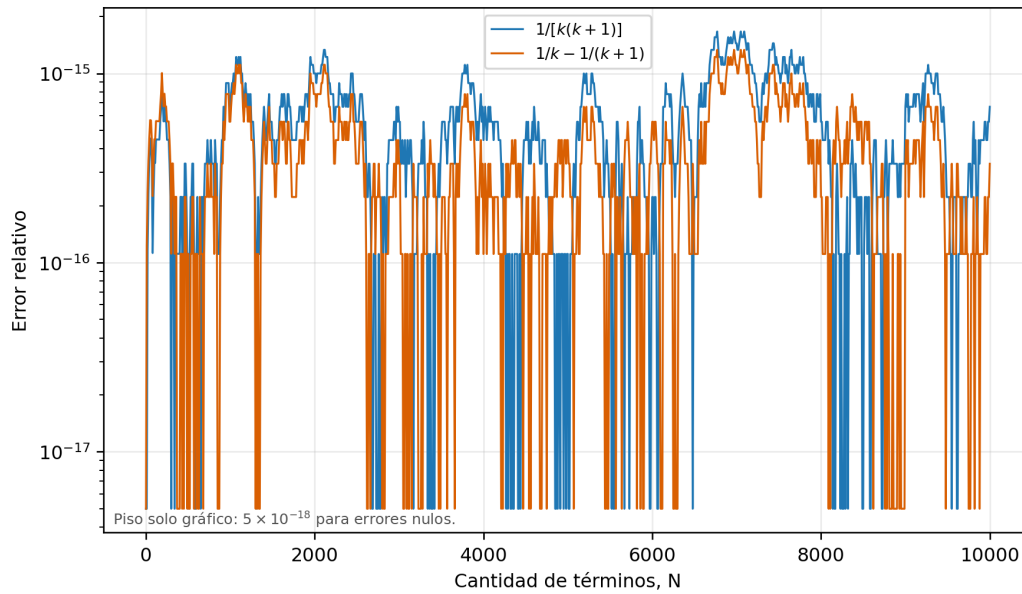


**Figura 6:** Error relativo de 30 permutaciones para  $N = 10^6$ . Cada punto usa una semilla distinta entre 20260914 y 20260943; el eje vertical es logarítmico.

### 4.3. Representaciones de $b_N$

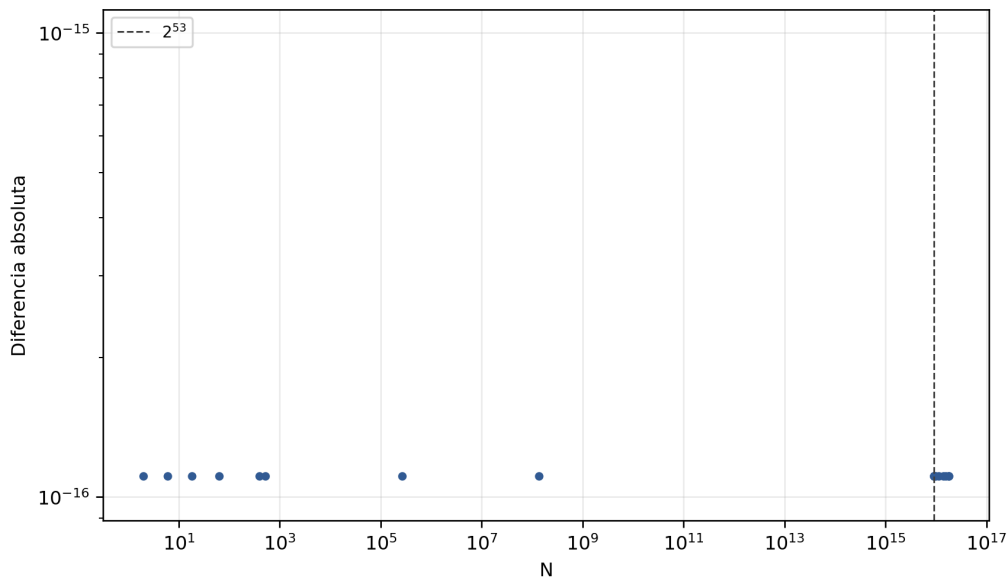
Las dos formas de la Ecuación 10 tuvieron errores cercanos al épsilon de máquina, pero no coincidieron para todos los  $N$ . Ninguna fue uniformemente mejor. Para  $N = 10000$ , los errores fueron  $6,66 \times 10^{-16}$  en la forma directa y  $3,33 \times 10^{-16}$  en la telescópica. La alternancia y los ceros

del rango completo se muestran en la Figura 7.



**Figura 7:** Error relativo de las dos sumatorias de  $b_N$  para  $N = 1, 10, 20, \dots, 10000$ . Los ceros originales se muestran en el piso gráfico  $5 \times 10^{-18}$ .

Las formas cerradas también difirieron por una ulp en ciertos valores. Para  $N = 2^{53}$ ,  $1 - 1/(N + 1)$  dio 0,9999999999999999, mientras  $N/(N + 1)$  se redondeó a 1. La Figura 8 ubica las diferencias no nulas y marca  $2^{53}$ .



**Figura 8:** Diferencias no nulas entre  $1 - 1/(N + 1)$  y  $N/(N + 1)$  hasta  $10^{18}$ . Ambos ejes son logarítmicos; la línea discontinua marca  $N = 2^{53}$ . Los valores sin diferencia no se dibujan.

#### 4.4. Representaciones de $c_N$

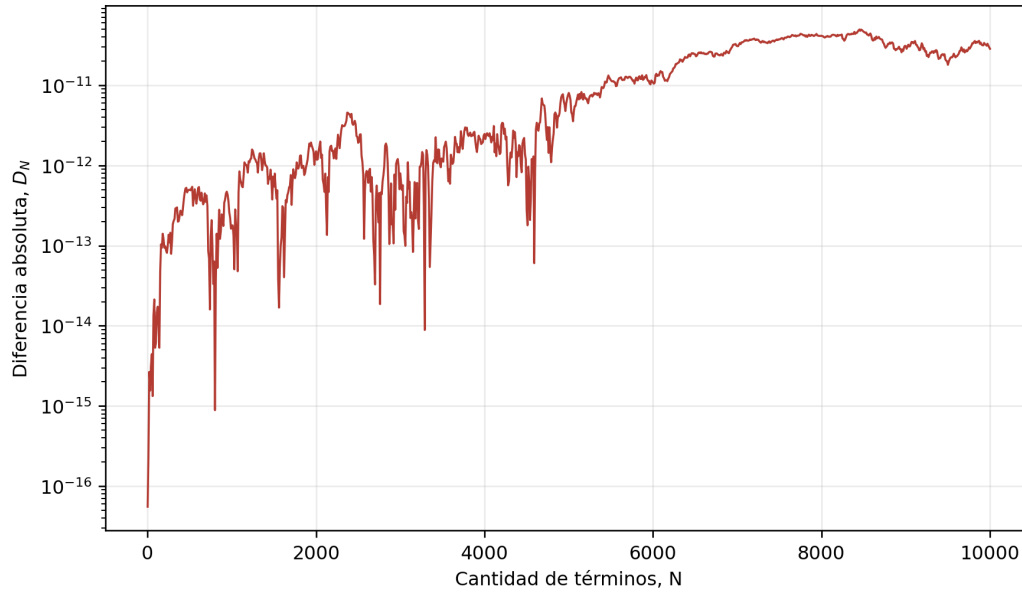
La diferencia absoluta entre las dos formas de  $c_N$  aumentó en términos generales, aunque presentó compensaciones locales. La Tabla 4 contiene valores tomados del CSV y la Figura 9 muestra el rango completo.

**Cuadro 4:** Valores de las dos representaciones de  $c_N$  y diferencia absoluta.

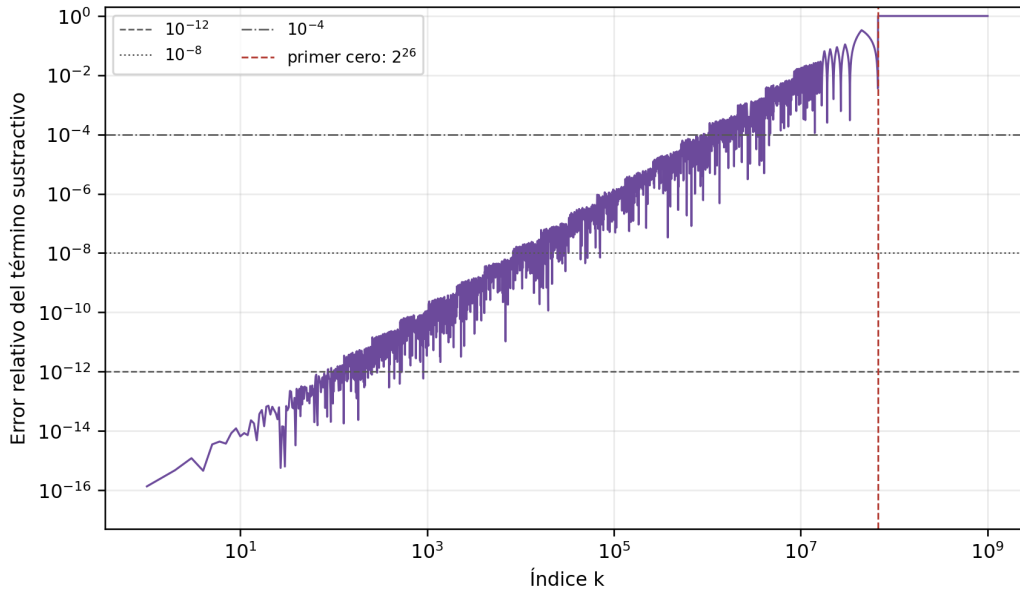
| $N$   | $c_N^{(1)}$ racionalizada | $c_N^{(2)}$ sustractiva | $D_N$                  |
|-------|---------------------------|-------------------------|------------------------|
| 1     | 0,4142135624              | 0,4142135624            | $5,55 \times 10^{-17}$ |
| 10    | 1,3560331890              | 1,3560331890            | $2,22 \times 10^{-16}$ |
| 100   | 2,4846796649              | 2,4846796649            | $6,22 \times 10^{-15}$ |
| 1000  | 3,6337202111              | 3,6337202111            | $1,79 \times 10^{-13}$ |
| 10000 | 4,7847877371              | 4,7847877371            | $2,85 \times 10^{-11}$ |

*Nota.* Las columnas centrales se muestran con diez decimales;  $D_N$  fue calculada con los valores completos.

Fuente: `datos/representaciones.csv`.

**Figura 9:** Diferencia absoluta  $D_N$  entre las dos sumatorias de  $c_N$  para  $N = 1, 10, 20, \dots, 10000$ . El eje vertical es logarítmico.

El error relativo del término sustractivo alcanzó por primera vez  $10^{-12}$  en  $k = 87$ ,  $10^{-8}$  en  $k = 8193$  y  $10^{-4}$  en  $k = 860284$ . El primer cero apareció en  $k = 67108864 = 2^{26}$ , aunque el término matemático y la forma racionalizada seguían siendo positivos (Figura 10).



**Figura 10:** Error relativo de  $\sqrt{k^2 + 1} - k$  respecto de la forma racionalizada. Las líneas horizontales indican los tres umbrales y la vertical marca el primer término sustractivo redondeado a cero. Ambos ejes son logarítmicos.

## 5. Discusión

### 5.1. Pérdida de la unidad, infinito y NaN

El contraste entre `int` y `float` se explica por su representación. Los enteros de Python tienen precisión arbitraria y crecen mientras haya memoria disponible ([Python Software Foundation, 2026a](#)); por eso el mismo valor exacto de  $b^k$  se suma y se resta. En `binary64`, la precisión es fija. Para  $b = 2$ ,  $(1 + 2^{53}) - 2^{53}$  vale cero numéricamente porque  $2^{53} + 1$  se redondea a  $2^{53}$ . En la tercera asociación,  $2^{53} - 1$  aún es representable y se conserva un término adicional; desde el paso siguiente también se pierde la unidad.

La asociación  $1 + (b^k - b^k)$  es más estable porque cancela dos representaciones idénticas antes de incorporar la unidad. Su límite no es la precisión del significando sino el rango del exponente. Cuando la potencia excede el máximo finito se obtiene infinito; después,  $\infty - \infty$  es una operación indeterminada y produce NaN. Como cualquier operación posterior con ese valor sigue dando NaN, las curvas quedan interrumpidas.

### 5.2. Bases negativas, $b \leq 1$ y enteros fijos

Para  $0 \leq b \leq 1$ , la magnitud de  $b^k$  no crece y la unidad no queda por debajo de su resolución. El mismo razonamiento vale para  $-1 \leq b < 0$ . Sin embargo, la condición literal  $b \leq 1$  también incluye  $b < -1$ . Allí  $|b|^k$  crece y el fenómeno reaparece. La alternancia de signo cambia si cada asociación exterior suma o resta la unidad respecto de una potencia positiva o negativa; por eso  $b = -2$  intercambia los resultados de  $b = 2$ .

Los enteros fijos muestran otro mecanismo. Sus potencias desbordan, pero bajo aritmética modular las identidades  $(1 + x) - x \equiv 1$  se conservan. El problema visible en `int8` aparece al acumular 1000 unos: 1000 módulo 256 es 232, interpretado con signo como  $-24$ . En otros lenguajes el desbordamiento puede lanzar una excepción, envolver modularmente o quedar sin

comportamiento definido; no debe generalizarse el comportamiento particular de NumPy.

### 5.3. Orden, Kahan y variabilidad aleatoria

Los términos  $1/[k(k+1)]$  son positivos y decrecientes. En el orden natural el acumulador se aproxima rápidamente a 1; los términos finales son demasiado pequeños respecto de su ulp y parte de su contribución se pierde. En el orden inverso, los términos pequeños se combinan cuando el acumulador también es pequeño. Esto explica la diferencia de más de dos órdenes de magnitud observada para  $N = 10^6$ .

Kahan consigue una precisión semejante sin invertir la secuencia. La compensación de la Ecuación 6 conserva una estimación de lo descartado y la reincorpora en iteraciones siguientes. El cero medido no prueba exactitud real: significa que el resultado final coincide con el `float64` de  $N/(N+1)$ .

Cada permutación aleatoria define una secuencia distinta de acumuladores y redondeos. Por eso los mismos términos generaron 30 errores diferentes. La semilla vuelve reproducible cada permutación, pero no elimina la variabilidad cuando cambia el orden. El rango observado también muestra que una permutación casual puede superar o empeorar el orden natural sin ofrecer una garantía sistemática.

### 5.4. Representaciones de $b_N$

La forma directa de cada término realiza una multiplicación y una división. La forma telescópica realiza dos divisiones y una resta de valores cercanos. Sus redondeos intermedios son diferentes, por lo que no hay obligación de coincidencia bit a bit. En este rango, ambos errores quedaron cerca del épsilon y se alternaron: el experimento no respalda declarar una de las dos sumatorias uniformemente superior.

El bonus cerrado revela otra sutileza. Cerca de  $2^{53}$ , convertir  $N$  y  $N+1$  a `float64` puede eliminar su diferencia antes de dividir. En  $N/(N+1)$  ambos se redondearon al mismo valor y el cociente fue 1. En  $1 - 1/(N+1)$  la pequeña corrección todavía produjo el `float64` inmediatamente inferior a 1. Con valores aún mayores, esa corrección cae por debajo de media ulp y las dos formas terminan en 1.

### 5.5. Cancelación en $c_N$

Para  $k$  grande,

$$\sqrt{k^2 + 1} = k\sqrt{1 + k^{-2}} \approx k + \frac{1}{2k}. \quad (12)$$

La forma sustractiva intenta recuperar una cantidad cercana a  $1/(2k)$  restando dos operandos de tamaño  $k$ . Los dígitos principales se cancelan y los errores de la raíz quedan amplificados. La forma racionalizada calcula directamente una cantidad pequeña y positiva; mantiene la escala del resultado y evita la resta problemática.

Los umbrales confirmados muestran una progresión coherente: desde  $k = 8193$  la forma sustractiva ya perdió al menos ocho cifras relativas. En  $2^{26}$ , el cálculo de  $k^2 + 1$  y su raíz se redondeó de modo que la raíz resultó igual a  $k$ ; la resta dio cero y eliminó por completo un término positivo. Las oscilaciones de  $D_N$  no contradicen la inestabilidad, porque errores sucesivos pueden tener signos opuestos y compensarse parcialmente.

## 5.6. Limitaciones

Los resultados corresponden al entorno documentado. IEEE 754 favorece la portabilidad, pero el último bit de `sqrt` o de ciertas optimizaciones puede cambiar entre bibliotecas. Además,  $N/(N+1)$  también es una aproximación `float64`; el error relativo compara dos números en la misma precisión. Una validación futura podría utilizar racionales exactos o precisión múltiple.

Para  $c_N$ , la consigna solicita comparar dos implementaciones, no calcular el error verdadero de cada una. La racionalización permite identificar la forma más estable, pero una referencia de alta precisión cuantificaría ambos errores por separado. Por último, 30 permutaciones prueban variabilidad, aunque no describen la distribución de todos los  $N!$  órdenes posibles.

## 6. Conclusiones

Los experimentos cumplieron el objetivo de mostrar que las propiedades algebraicas no se trasladan automáticamente a la aritmética computacional. La precisión finita obliga a redondear cada resultado intermedio y hace que el tipo, la asociación y el orden formen parte efectiva del algoritmo.

En asociatividad, los enteros arbitrarios de Python preservaron  $a_N = N$ , mientras `float64` perdió unidades y luego produjo valores no finitos. La asociación que cancela primero  $b^k - b^k$  fue más estable hasta el desbordamiento. Las bases negativas demostraron que importa el crecimiento de  $|b|$ , no solo la desigualdad  $b \leq 1$ , y los tipos fijos mostraron que la exactitud entera depende de no exceder el rango disponible.

En la suma de  $b_N$ , procesar de menor a mayor y usar Kahan redujo el error frente al orden natural. Las permutaciones aleatorias confirmaron que los mismos sumandos pueden dar distintos resultados. Las dos representaciones sumatorias de  $b_N$  quedaron cerca de la precisión de máquina sin una ganadora uniforme, mientras la forma sustractiva de  $c_N$  sufrió cancelación catastrófica y llegó a producir términos nulos.

En consecuencia, aumentar  $N$  no garantiza una aproximación mejor si el algoritmo descarta información. Una implementación confiable debe elegir conscientemente el tipo numérico, el orden y la forma algebraica. Como línea futura, conviene repetir el estudio con `float32`, `longdouble`, precisión arbitraria y suma por pares, comparando precisión, memoria y tiempo.

## Referencias

- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48. <https://doi.org/10.1145/103162.103163>.
- Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition. <https://doi.org/10.1137/1.9780898718027>.
- Institute of Electrical and Electronics Engineers (2019). IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). <https://doi.org/10.1109/IEEESTD.2019.8766229>.
- Kahan, W. (1965). Pracniques: Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1):40. <https://doi.org/10.1145/363707.363723>.
- NumPy Developers (2026). Data types and integer overflow. NumPy documentation. Recuperado el 12 de septiembre de 2026 de <https://numpy.org/doc/stable/user/basics.types.html>.
- Python Software Foundation (2026a). Built-in types: Numeric types - int, float, complex. Python 3.14.7 documentation. Recuperado el 12 de septiembre de 2026 de <https://docs.python.org/3/library/stdtypes.html>.
- Python Software Foundation (2026b). Floating-point arithmetic: Issues and limitations. Python 3.14.7 documentation. Recuperado el 12 de septiembre de 2026 de <https://docs.python.org/3/tutorial/float.html>.
- Python Software Foundation (2026c). random - generate pseudo-random numbers. Python 3.14.7 documentation. Recuperado el 12 de septiembre de 2026 de <https://docs.python.org/3/library/random.html>.